

SOFTWARE ENGINEERING

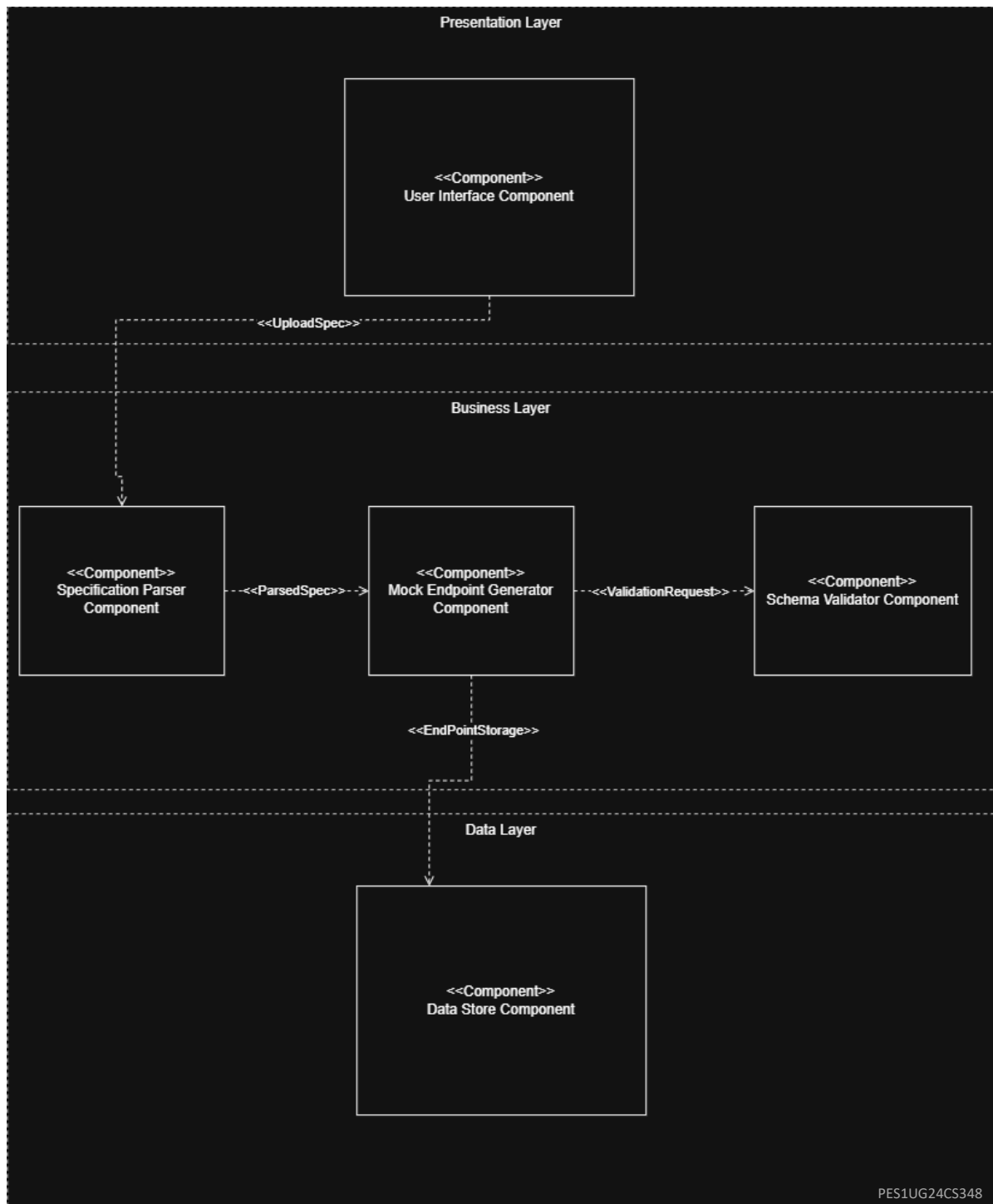
Lab 3: Component Modelling & Architectural Pattern Selection

Name: Puneeth Yenneti

SRN: PES1UG24CS348

Section: F

Component Diagram: (Problem Statement: OpenAPI Mock Server Generator System)



Justification:

So my architectural choice is a **Layered Architecture**.

Layered Architecture organises the system into three distinct horizontal layers – presentation, business and data, where each layer only communicates with the layer directly below it.

The Reason for this choice is –

- the OpenAPI mock server generator has three distinct responsibilities: accepting file uploads from user (Presentation), processing and generating endpoints (Business) and storing specification data and logs (Data). These map directly and cleanly onto 3 layers. making this architecture intuitively better.
- the system only serves 2 actors (API developer and QA Engineer) through 1 interface. Any other Architectures may introduce unnecessary operational complexity for a tool of this scale (Microservices architecture) and may not adequately separate the business logic from data logic (Client-Server Architectue)
- The layered approach ensures that all uploaded OpenAPI specification files pass through the Schema Validator Component in the Business Layer before any data reaches the Data Store. This prevents malformed or malicious YAML/JSON files from being persisted or executed, acting as a mandatory validation checkpoint between external input and internal storage.
- The Data Store Component in the Data Layer can cache previously parsed specifications and their generated endpoint configurations. This means repeated mock API requests from the QA Engineer do not re-trigger the full parsing pipeline in the Business Layer, significantly reducing response latency during load testing workflows.

Thus Layered Architecture is the most suitable choice for the OpenAPI Mock Server Generator System. It aligns naturally with the system's structure, cleanly separates the upload, processing, and storage responsibilities, and provides clear security and performance benefits without introducing unnecessary complexity.

The three-layer design ensures that both the API Developer and QA Engineer can interact with a reliable, validated, and responsive mock server tool.